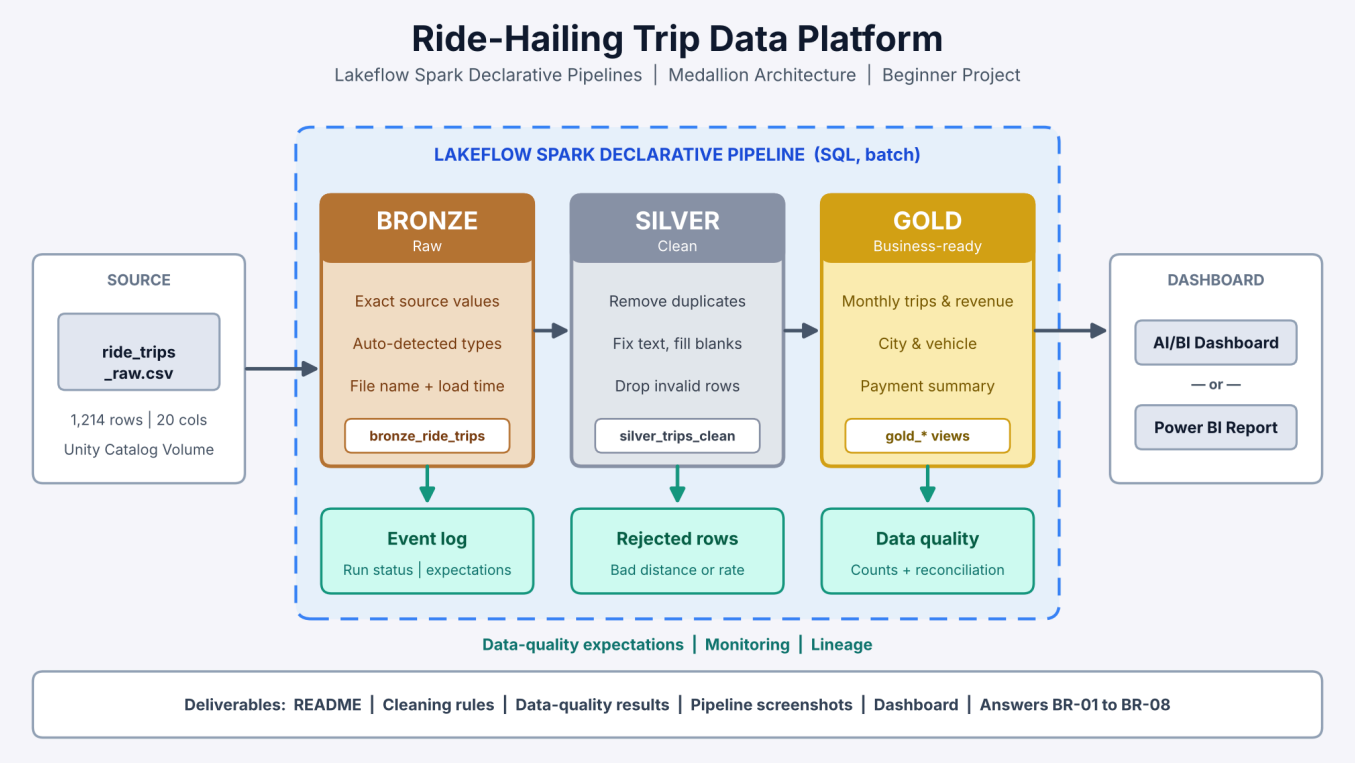
INNOMATICS RESEARCH LABS

Lakeflow Spark Declarative Pipelines

Ride-Hailing Trip Analytics Platform



Project level: Beginner • Estimated time: 6–8 hours • SQL you need: SELECT, WHERE, GROUP BY, COUNT, SUM, AVG, DISTINCT, TRIM, INITCAP, COALESCE, CASE WHEN.

1. Executive Summary

A cab-booking company (like a city ride-hailing app) exports its completed trips to a CSV file every month. Managers want simple, reliable reports on trips, revenue, cities, vehicle types and payment methods. The file has common problems: extra spaces, inconsistent capital letters, blank values, duplicate trips, and a few impossible distances and rates. You will build a Databricks Medallion Architecture that keeps the raw file, cleans it step by step, creates business-ready summary tables, and shows them in a dashboard.

Project goal: Transform a raw cab trips CSV into trusted Gold-layer tables using Databricks SQL and Lakeflow Spark Declarative Pipelines, then build one dashboard on top of them.

2. Business Background

The company runs cabs in eight Indian cities across four regions. Customers book a Bike, Auto, Mini, Sedan or Premium ride through the App, the Website or the Call Center. Each trip has a distance, a rate per km, a fare, an optional discount, a payment method (UPI, Cash, Card or Wallet) and a customer rating. The driver receives a payout, and the rest is the company's earnings.

Current business challenges

- City, vehicle type and booking channel values have extra spaces and mixed capital letters.
- Some trips are missing the city, payment method, customer ID, fare or rating.
- Some trips appear twice, which makes trips and revenue look bigger than they are.
- A few trips have a zero or negative distance, or a negative rate per km.
- Reports are made by hand in Excel every month, which is slow and error-prone.

3. Business Problem and Objectives

The company has no automatic way to turn the raw trips file into numbers it can trust. This project builds a repeatable pipeline that keeps the original file, cleans the data, prepares summary tables and shows them in a dashboard.

Project objectives

- Load the trips CSV into Databricks from a Unity Catalog Volume.
- Keep the raw data in a Bronze table.
- Clean the data in a Silver table and keep the bad trips in a separate rejected table.
- Create Gold summary tables for monthly trips, cities, vehicle types and payment methods.
- Count the data problems and check that no row was lost.
- Build one dashboard in Databricks AI/BI or Power BI.

4. Dataset Overview

The learning dataset ride_trips_raw.csv is synthetic (made up for this project). It has 1,214 rows and 20 columns covering cab trips in 2025. It looks like a real ride-hailing export but contains no real customer data.

Ride Trips Dataset :- Click Here to Download Ride Trips Dataset

Dataset attribute	Description	Why it matters
Grain	One row = one completed trip. trip_id is unique for each real trip.	Tells you how to count trips and find duplicates
Trip fields	trip_id, trip_date, pickup_hour, trip_type, booking_channel	Supports monthly and booking analysis
Location fields	city, pickup_zone, region	Supports city and region reports
Vehicle & people fields	vehicle_type, driver_id, customer_id, customer_rating	Supports vehicle and service-quality reports

Dataset attribute	Description	Why it matters
Money fields	distance_km, rate_per_km, fare_amount, discount_amount, driver_payout, payment_method, payment_status	Supports revenue and payment reports
Planted problems	Extra spaces, mixed case, blanks, duplicate trips, zero/negative distance, negative rate	Gives you real cleaning practice

Column dictionary

Column	Example	Description
trip_id	TR200001	Unique ID of one trip
customer_id	CU3125	Customer who booked (sometimes blank)
driver_id	DR745	Driver who did the trip
trip_date	2025-06-18	Date of the trip
pickup_hour	18	Hour of pickup, 0–23
city	Mumbai	City of the trip (sometimes blank or messy)
pickup_zone	Andheri	Area where the trip started
region	West	South, West, North or East
vehicle_type	Sedan	Bike, Auto, Mini, Sedan, Premium (sometimes messy)
booking_channel	App	App, Website or Call Center (sometimes messy)
distance_km	12.4	Trip distance in km
duration_min	38	Trip time in minutes
rate_per_km	17.50	Price per km in ₹
fare_amount	217.00	distance_km × rate_per_km (sometimes blank)
discount_amount	21.70	Discount given to the customer
payment_method	UPI	UPI, Cash, Card or Wallet (sometimes blank)
payment_status	Paid	Paid or Pending
driver_payout	156.24	Amount paid to the driver
customer_rating	5	Rating from 1 to 5 (sometimes blank)
trip_type	City Ride	City Ride, Airport or Outstation

Key business formulas

- fare_amount = distance_km × rate_per_km
- net_revenue = fare_amount − discount_amount
- platform_earnings = net_revenue − driver_payout

Important: trip_id should be unique. If the same trip_id appears more than once, the extra rows are duplicates. customer_id and driver_id are NOT unique, because one customer or driver has many trips.

5. Key Terms for Beginners

Term	Simple meaning
Unity Catalog Volume	A folder inside Databricks where you upload files such as CSVs.
Medallion Architecture	Organising data in three layers: Bronze (raw), Silver (clean), Gold (summaries).
Lakeflow Spark Declarative Pipelines	A Databricks feature where you write SQL that says what each table should contain; Databricks works out the order and runs it.
Streaming table	A table that reads new files incrementally. We use it for Bronze.
Materialized view	A saved query result that the pipeline keeps up to date. We use it for Silver and Gold.
Expectation	A data-quality rule, e.g. distance_km > 0. With ON VIOLATION DROP ROW, bad rows are removed and counted.
NULL	An empty value. AVG() and SUM() simply skip NULLs, which is why a blank rating does not need to be replaced.
Reconciliation	Checking that every input row is accounted for: kept, rejected or removed as a duplicate.

6. Proposed Architecture and Data Flow

Bronze keeps the raw trips. Silver cleans them: good trips go to silver_trips_clean and bad trips go to silver_trips_rejected. Gold builds summary tables from the clean trips only, plus one data-quality table.

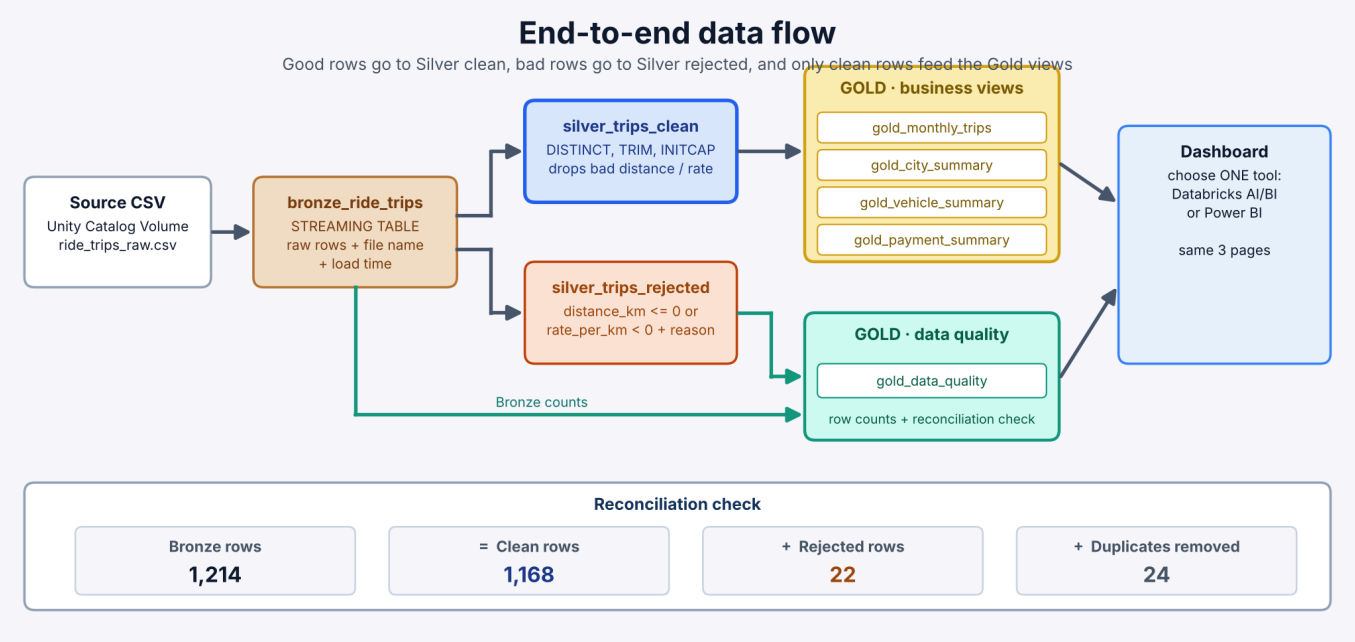


Figure 1 – End-to-end data flow and reconciliation check

7. Medallion Layer Responsibilities

Layer	Purpose	What you do	Result
Bronze	Keep the source data	Load the CSV as it is; add file name and load time	1 raw table
Silver	Clean the data	Remove duplicates, fix text, fill blanks, drop bad trips (and keep them in a rejected table)	1 clean table + 1 rejected table
Gold	Answer business questions	GROUP BY month, city, vehicle type and payment method; count data problems	5 summary tables
Dashboard	Show the results	Build 3 pages in Databricks AI/BI or Power BI	1 dashboard

8. Business Requirements

ID	Requirement	Business expectation
BR-01	Monthly trends	Show trips, total km, net revenue, driver payout and platform earnings for each month.
BR-02	City performance	Compare trips, net revenue and average fare per trip by region and city.
BR-03	Vehicle performance	Compare trips, km, net revenue and average customer rating by vehicle type.
BR-04	Payment analysis	Compare trips, net revenue and pending amount by payment method.
BR-05	Data-quality visibility	Show how many rows had each problem (duplicates, bad distance, bad rate, blanks).
BR-06	Reconciliation	Prove that Bronze rows = clean rows + rejected rows + duplicates removed.

ID	Requirement	Business expectation
BR-07	Repeatability	Run everything as a Databricks pipeline that can be re-run with one click.
BR-08	Dashboard	Build one dashboard (Databricks AI/BI or Power BI) on the Gold tables.

9. Data Cleaning Rules

Silver cleaning is done in four simple steps. Each step uses one SQL idea.

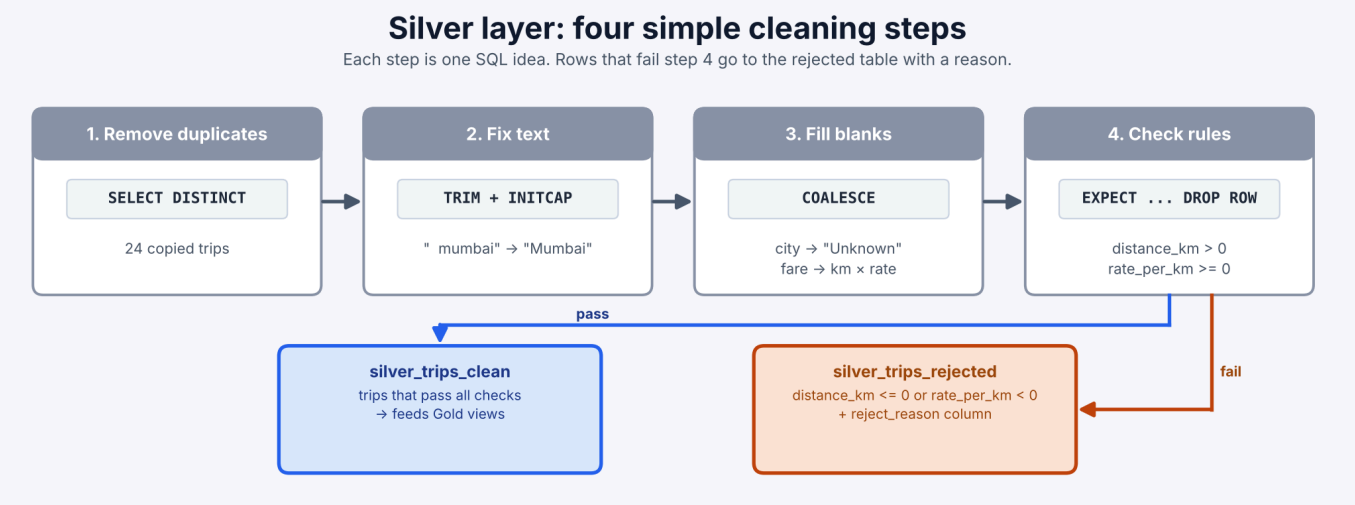


Figure 2 – The four Silver cleaning steps

Problem	Example	What to do	SQL hint
Duplicate trips	Same trip_id appears twice	Keep only one copy	SELECT DISTINCT
Extra spaces / mixed case	" mumbai", "SEDAN", " app "	Remove spaces, use Title Case	INITCAP(TRIM(col))
Blank city or payment method	city is empty	Replace with "Unknown"	COALESCE(col, 'Unknown')
Blank fare	fare_amount is empty	Calculate distance_km × rate_per_km	COALESCE(fare_amount, distance_km * rate_per_km)
Blank rating or customer ID	customer_rating is empty	Keep the row and leave it blank (AVG skips blanks)	No change
Bad distance or rate	distance_km = 0 or -1.5; rate_per_km = -14.20	Remove from clean table, keep in rejected table	EXPECT (distance_km > 0) ON VIOLATION DROP ROW

Watch out: Do not use INITCAP on payment_method. It would change "UPI" into "Upi". Payment method only needs COALESCE.

10. Project Scope

Included

- One synthetic cab trips CSV file.
- Batch loading from a Unity Catalog Volume.
- Bronze, Silver and Gold tables in one Lakeflow pipeline written in SQL.
- Simple data-quality checks and a reconciliation check.
- One dashboard in Databricks AI/BI or Power BI.

Not included

- Live GPS or real-time trip streaming.
- Cancelled trips, surge pricing or more than one source system.
- Real customer data and privacy controls.
- Advanced data modelling (dimensions, SCD) and production alerting.

Scope note This is a learning project. Keep it simple, make it work end to end, and explain your choices.

11. Implementation Workflow

01 • Understand	Read the business questions and the 20 columns. Open the CSV and spot the problems yourself.
02 • Prepare	Create a catalog, schema and Volume. Upload the CSV.
03 • Ingest	Create the Bronze streaming table and check that 1,214 rows were loaded.
04 • Clean	Create the Silver clean and rejected tables using the four cleaning steps.
05 • Curate	Create the four Gold business tables with GROUP BY.
06 • Validate	Create the Gold data-quality table and check the reconciliation.
07 • Publish & Visualize	Build one dashboard: Databricks AI/BI or Power BI.
08 • Explain	Present your pipeline, cleaning rules, results and dashboard.

Step-by-step guide with SQL hints

Use these starter snippets. Replace or complete the parts marked with "...".

Step 02 – Prepare the storage

```
CREATE CATALOG IF NOT EXISTS mobility_catalog;   -- or use a catalog you already have
CREATE SCHEMA  IF NOT EXISTS mobility_catalog.trips;
CREATE VOLUME  IF NOT EXISTS mobility_catalog.trips.raw_files;

-- Upload ride_trips_raw.csv into the volume (Catalog Explorer → raw_files → Upload)
-- Then preview it:
SELECT * FROM read_files('/Volumes/mobility_catalog/trips/raw_files/',
                         format => 'csv', header => true) LIMIT 10;
```

Step 03 – Create the pipeline and the Bronze table

Create a new ETL pipeline (Lakeflow Spark Declarative Pipelines), set the default catalog to mobility_catalog and schema to trips, and add a SQL file with:

```
CREATE OR REFRESH STREAMING TABLE bronze_ride_trips AS
SELECT *,
       _metadata.file_name AS source_file,
       current_timestamp() AS loaded_at
FROM STREAM read_files('/Volumes/mobility_catalog/trips/raw_files/',
                       format => 'csv', header => true);
```

Step 04 – Silver clean and rejected tables

```
CREATE OR REFRESH MATERIALIZED VIEW silver_trips_clean (
  CONSTRAINT valid_distance EXPECT (distance_km > 0)  ON VIOLATION DROP ROW,
  CONSTRAINT valid_rate     EXPECT (rate_per_km >= 0) ON VIOLATION DROP ROW
) AS
SELECT DISTINCT
  trip_id, customer_id, driver_id, trip_date,
  COALESCE(INITCAP(TRIM(city)), 'Unknown')          AS city,
  INITCAP(TRIM(vehicle_type))                       AS vehicle_type,
  COALESCE(fare_amount, distance_km * rate_per_km)  AS fare_amount,
  COALESCE(payment_method, 'Unknown')               AS payment_method,
  ...   -- add the other columns; clean booking_channel like vehicle_type
FROM bronze_ride_trips;

CREATE OR REFRESH MATERIALIZED VIEW silver_trips_rejected AS
SELECT *,
  CASE WHEN distance_km <= 0 THEN 'Distance is zero or negative'
       ELSE 'Rate per km is negative' END AS reject_reason
FROM bronze_ride_trips
WHERE distance_km <= 0 OR rate_per_km < 0;
```

Step 05 – Gold business tables

```
CREATE OR REFRESH MATERIALIZED VIEW gold_monthly_trips AS
SELECT DATE_TRUNC('MONTH', trip_date)           AS trip_month,
       COUNT(*)                                 AS trips,
       SUM(distance_km)                         AS total_km,
       SUM(fare_amount - discount_amount)       AS net_revenue,
       SUM(driver_payout)                       AS driver_payout,
       SUM(fare_amount - discount_amount) - SUM(driver_payout) AS platform_earnings
FROM silver_trips_clean
GROUP BY DATE_TRUNC('MONTH', trip_date);
```

Build the other three Gold tables with the same pattern:

Gold table	GROUP BY	Columns to calculate
gold_city_summary	region, city	trips, net_revenue, avg_fare_per_trip (AVG of fare − discount)
gold_vehicle_summary	vehicle_type	trips, total_km, net_revenue, avg_rating (AVG of customer_rating)
gold_payment_summary	payment_method	trips, net_revenue, pending_amount (SUM with CASE WHEN payment_status = 'Pending')

Step 06 – Data quality and reconciliation

```
CREATE OR REFRESH MATERIALIZED VIEW gold_data_quality AS
SELECT COUNT(*)                                          AS bronze_rows,
       COUNT(*) - COUNT(DISTINCT trip_id)                AS duplicate_rows,
       SUM(CASE WHEN distance_km <= 0 THEN 1 ELSE 0 END)    AS invalid_distance_rows,
       SUM(CASE WHEN city IS NULL THEN 1 ELSE 0 END)        AS missing_city_rows,
       ...   -- add negative rate, missing payment, missing fare, missing rating
       (SELECT COUNT(*) FROM silver_trips_clean)          AS clean_rows,
       (SELECT COUNT(*) FROM silver_trips_rejected)       AS rejected_rows
FROM bronze_ride_trips;
```

Then run the pipeline (Start), open the pipeline graph, and check: **bronze_rows = clean_rows + rejected_rows + duplicate_rows**.

12. Dashboard (Choose Databricks AI/BI or Power BI)

The dashboard is the last stage of the project. It reads only the Gold tables. Build it in **one** tool only: Option A or Option B. Both give the same three pages.

Choose one option: If you build the dashboard in Power BI, skip Option A. If you build it as a Databricks AI/BI dashboard, skip Option B.

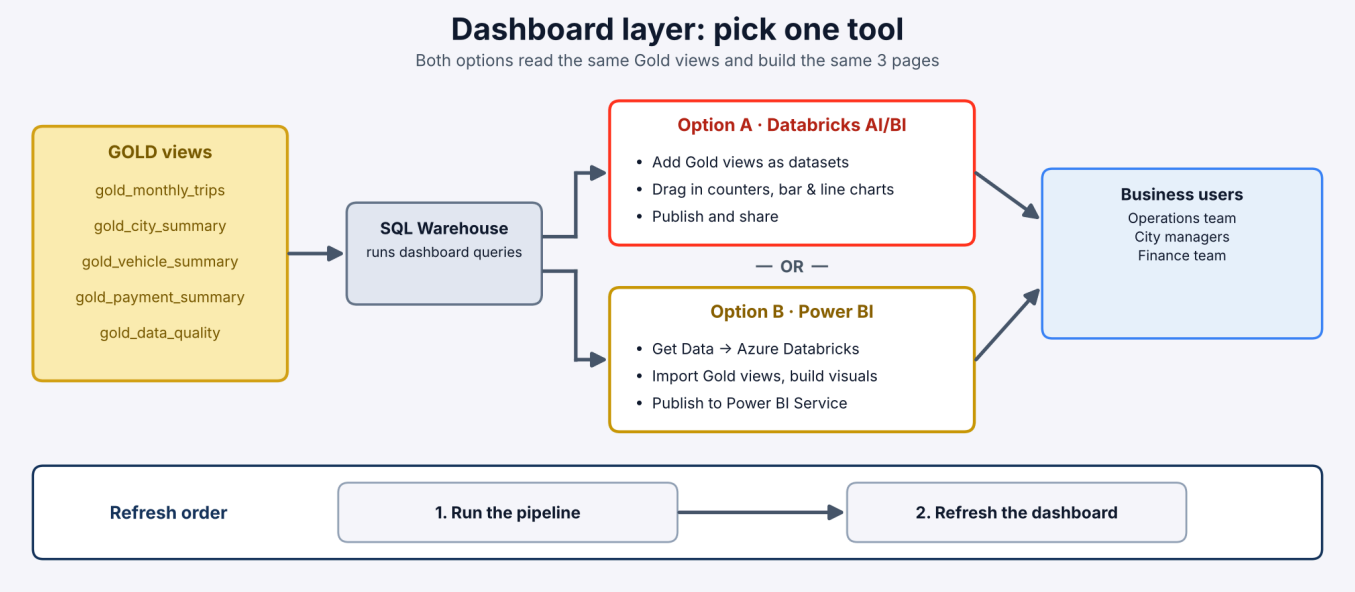


Figure 3 – Dashboard layer: one tool, same Gold tables

Dashboard pages (same for both options)

Page	Visuals	Gold table
1. Overview	KPI cards: Total Trips, Net Revenue, Total Distance, Platform Earnings; line chart of net revenue by month; bar chart of net revenue by city	gold_monthly_trips, gold_city_summary
2. Vehicles & Payments	Bar chart of trips and revenue by vehicle type; bar chart of average rating by vehicle type; bar chart of trips by payment method; pending amount card	gold_vehicle_summary, gold_payment_summary
3. Data Quality	Cards: Bronze rows, clean trips, rejected rows, duplicates; reconciliation passed (Yes/No); bar chart of problem counts	gold_data_quality



Figure 4 – Example Overview page (numbers from the reference solution)

Option A – Databricks AI/BI Dashboard

- **Create:** in Databricks choose New → Dashboard.
- **Add data:** on the Data tab, add each Gold table as a dataset (pick the table or write SELECT * FROM ...).
- **Add visuals:** on the canvas, add Counter widgets for the KPIs (sum of trips, net_revenue, total_km, platform_earnings), a Line chart for monthly revenue and Bar charts for city, vehicle type, rating and payment method.
- **Add filters:** add a filter for month, region or vehicle type where those columns exist.

- **Publish:** click Publish and share the dashboard with your team. Optionally add a refresh schedule.

Option B – Power BI Report

- **Connect:** in Power BI Desktop choose Get Data → Azure Databricks. Copy the Server hostname and HTTP path from your SQL warehouse (Connection details tab) and sign in.
- **Load data:** open mobility_catalog → trips, tick the five gold_* tables, and choose Import.
- **Add visuals:** use Card visuals for the KPIs (Sum of trips, net_revenue, total_km, platform_earnings), a Line chart for monthly revenue and Bar charts for city, vehicle type and payment method.
- **Ratings:** show avg_rating by vehicle type in a bar chart or table on page 2 (do not add up or average the averages).
- **Add filters:** add Slicers for month, region or vehicle type on the pages where those columns exist.
- **Publish:** click Publish to send the report to Power BI Service and share it. Optionally set a scheduled refresh.

Check your numbers: The Total Trips card should equal the clean_rows value in gold_data_quality (1,168). If it does not, check your visual settings.

13. Expected Deliverables

- A pipeline with Bronze, Silver (clean + rejected) and Gold tables.
- A short list of your cleaning rules and why you chose them.
- The gold_data_quality result showing the reconciliation check passed.
- One dashboard with three pages (Databricks AI/BI or Power BI).
- A README explaining how to set up and run the project.
- Screenshots of the pipeline graph, the Gold tables and the dashboard.

14. Acceptance Criteria / Definition of Done

- The CSV loads from the Volume and Bronze has 1,214 rows.
- Silver clean has no duplicate trips, no extra spaces, no blank city or payment method, and no bad distance or rate.
- Rejected trips are saved with a reason.
- The four Gold business tables answer BR-01 to BR-04.
- gold_data_quality shows that bronze rows = clean + rejected + duplicates.
- The pipeline runs again with one click and gives the same result.
- The dashboard shows the three pages and its numbers match the Gold tables.

15. Questions for Project Review

1. Why do we keep the raw data in Bronze instead of cleaning the file directly?
2. What is the difference between removing a trip and putting it in the rejected table?
3. Why can trip_id find duplicates but customer_id cannot?
4. Why do we use TRIM before INITCAP, and why not on payment_method?
5. Why do we leave a blank rating as NULL instead of replacing it with 0?
6. How does the reconciliation check prove that no row was lost?
7. Which vehicle type earns the most, and which has the best rating?

8. Which dashboard tool did you choose, and why?

16. Final Business Outcome

At the end of the project, the company has a repeatable pipeline that turns a messy trips file into clean, trusted summary tables and a simple dashboard. You can explain where the data came from, what problems you found, how you fixed them, and how you proved the numbers are correct.

Key takeaway Medallion Architecture means each layer makes the data a little more trustworthy: Bronze keeps it, Silver cleans it, Gold summarises it, and the dashboard shares it.